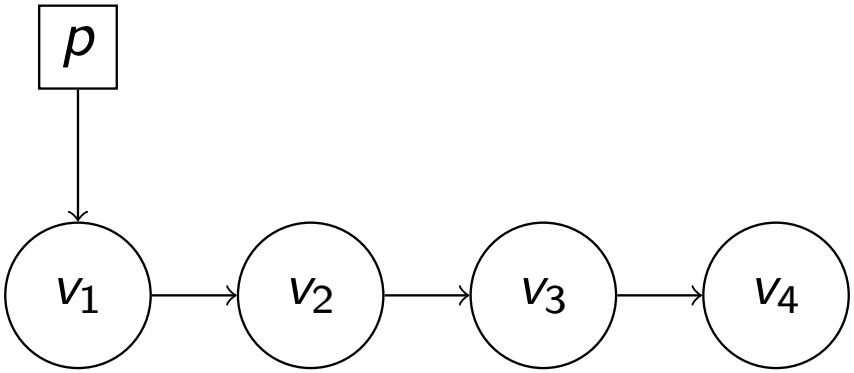
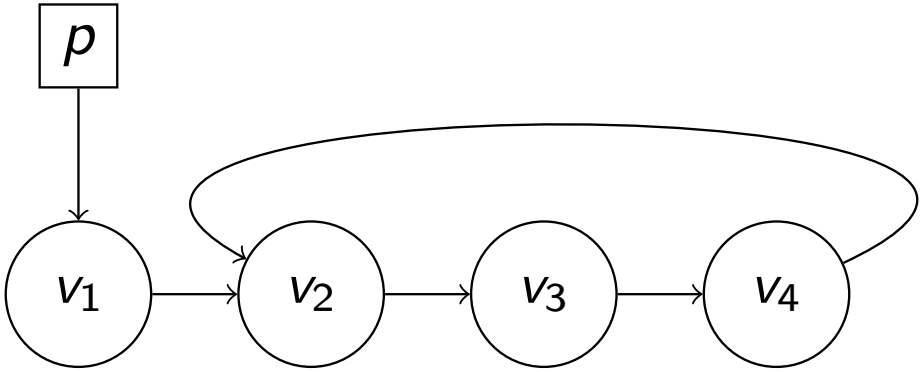
Single linked list



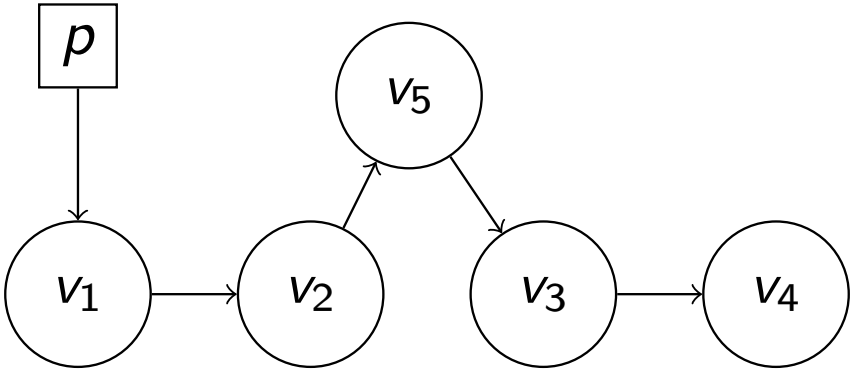

```
class List {
        int     data;
        List    next;
};

List    p;
```

A corrupted single linked list



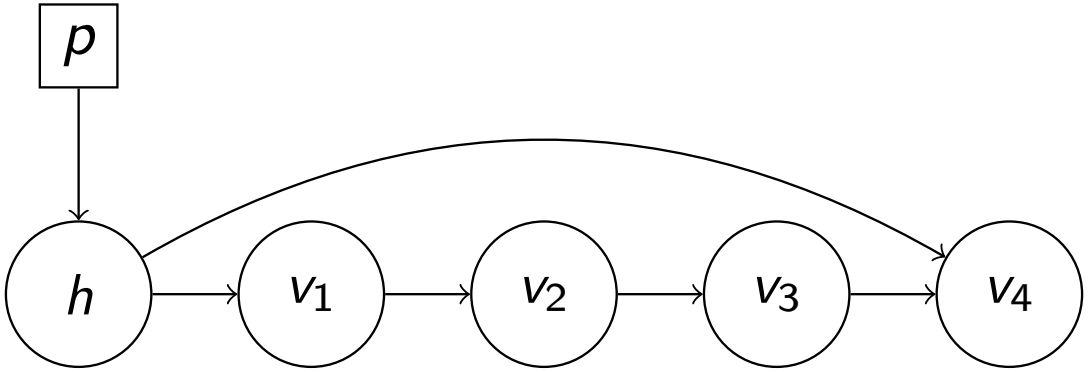
- Quiz: how can you check if a list is corrupted without looping forever?

Inserting a new node



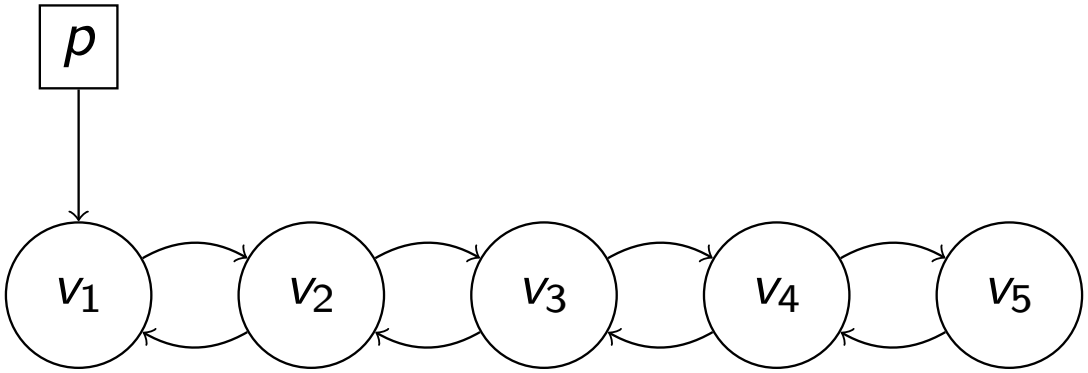
- Lists are more flexible than arrays

Optimizing append



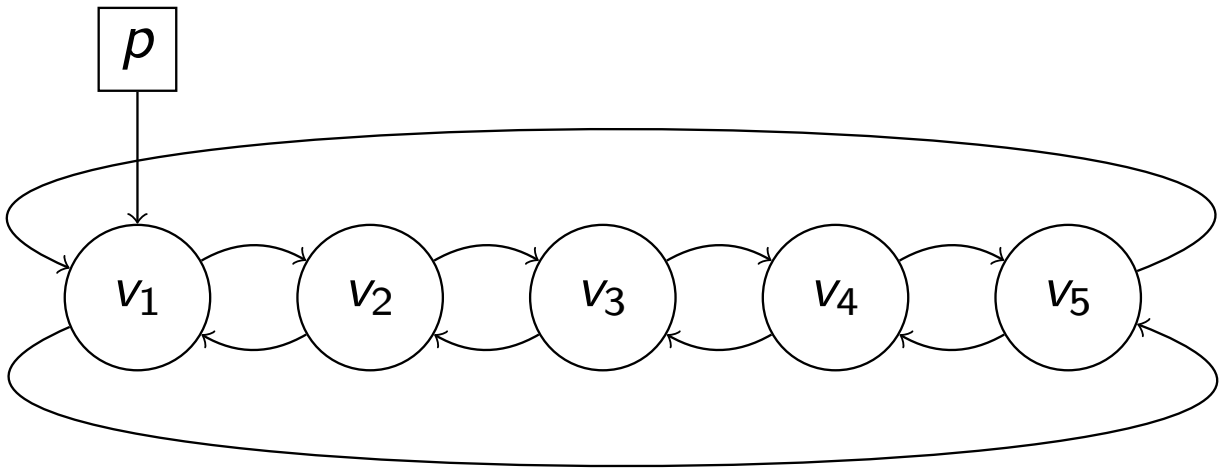
- A header node with pointers to both first and last nodes

Double linked list



- More efficient in some situations

A circular double linked list



- Beware of infinite loops!
- Often a do-while loop is convenient

Binary search tree

- A tree node t
- $left(t) = null$ or $key(left(t)) < key(t)$
- $right(t) = null$ or $key(right(t)) > key(t)$
- to insert a (key,value) pair,
- to delete a node with a certain key, and
- to search for a node with a certain key.

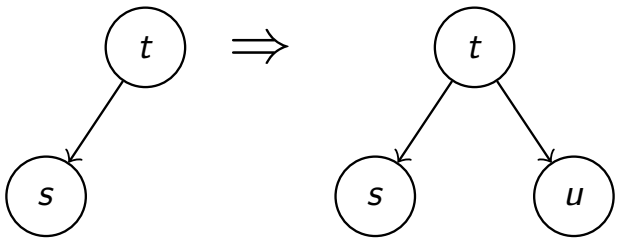
Balanced binary search trees

- Without balancing, the running time of insert, search, and delete would be $O(n)$
- Two Russian mathematicians, Georgy Adelson-Velsky and Evgenii Landis, discovered in 1962 the first self-balancing binary search tree with $O(\log n)$ time for insert, delete, and search: the AVL-tree.
- In 1972 the German computer scientist Rudolf Bayer invented another self-balancing search tree: the red-black tree, with the same time complexity

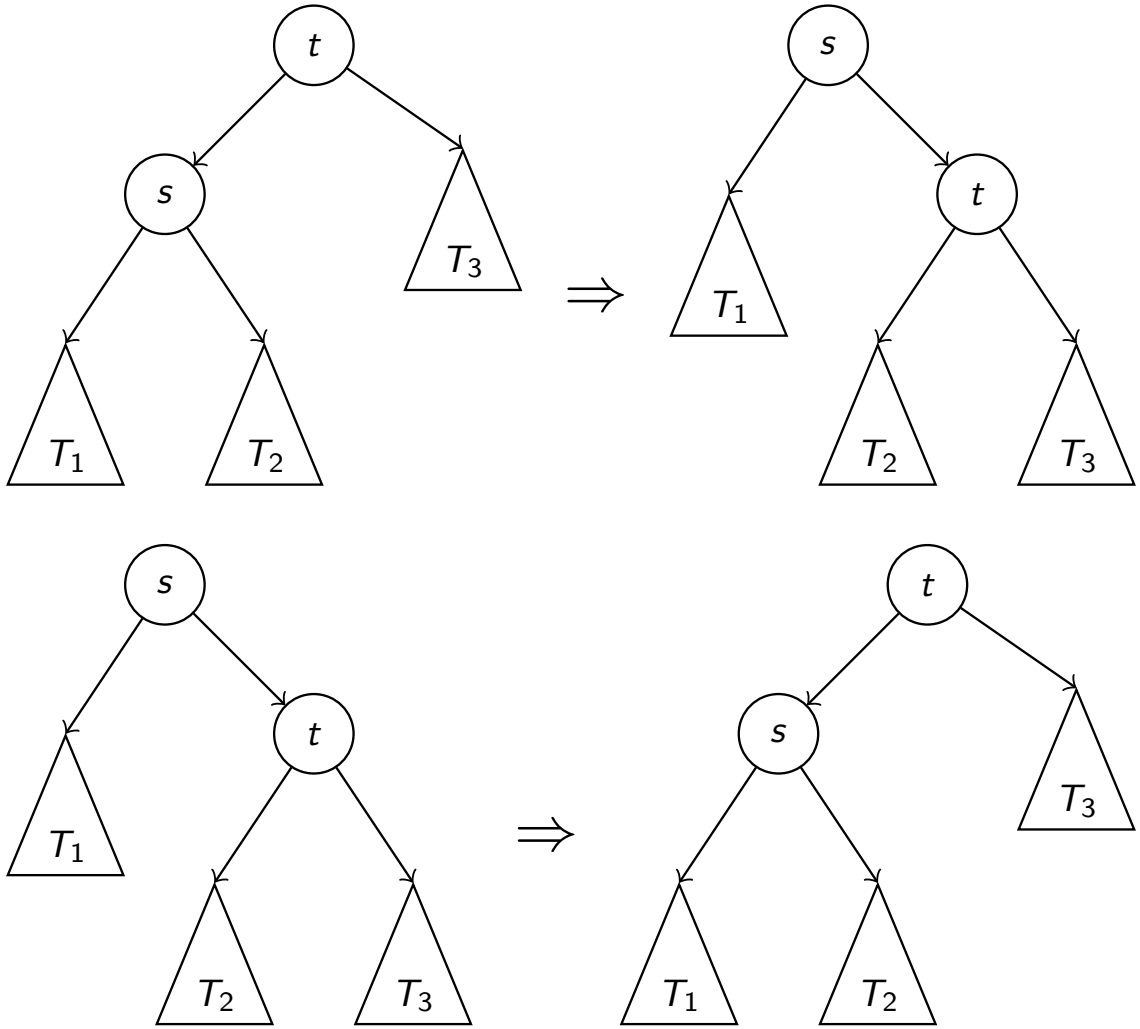
AVL tree balance attribute

balance	meaning
-1	left subtree is one higher than right subtree
0	left and right subtrees have equal heights
1	right subtree is one higher than left subtree

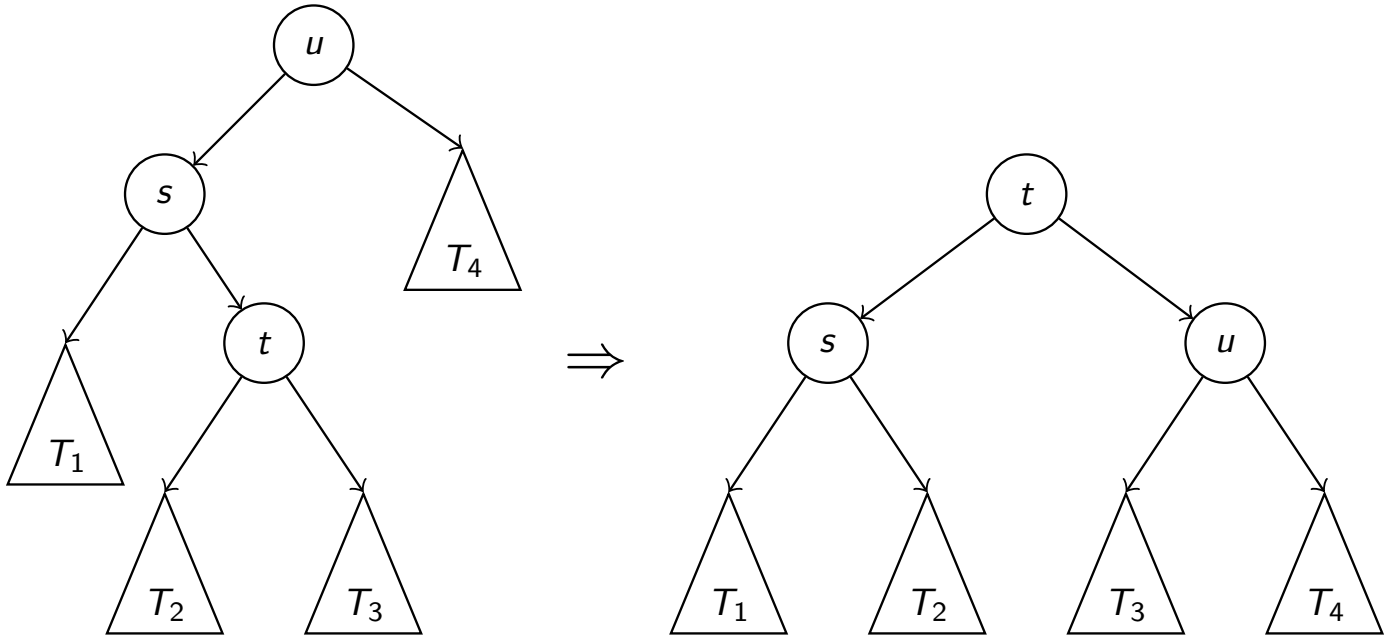
Insertion

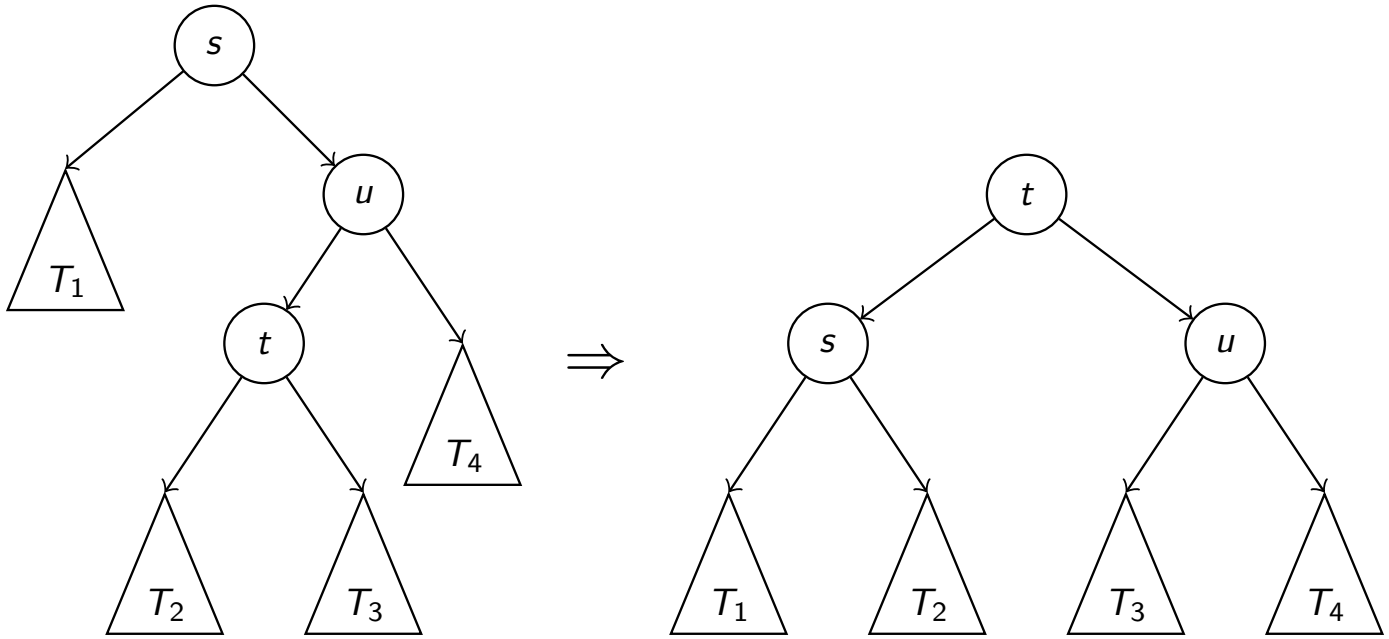


Single rotations



Double rotations

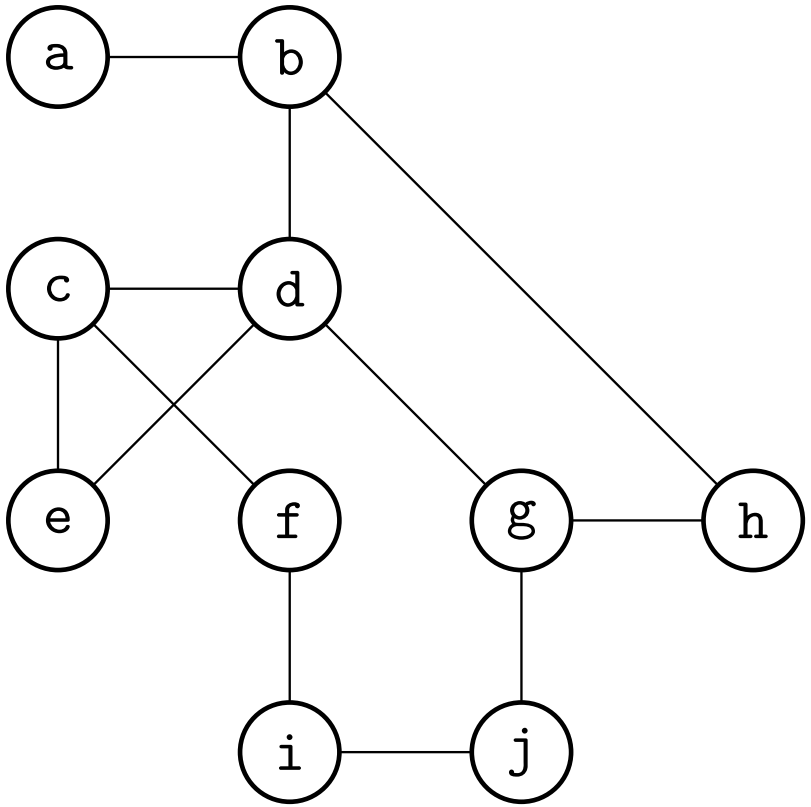




Graphs

- Notation
- Graph traversal and connectivity
- Testing bipartiteness
- Connectivity in directed graphs

Graphs



- $G = (V, E)$
- V is a set of nodes or vertices
- E is a set of edges or arcs
- $V = \{a, b, c, d, e, f, g, h, i, j\}$
- $E = \{a - b, b - d, ..., i - j\}$, or
- $E = \{(a, b), (b, d), ..., (i, j)\}$
- $n = |V|$
- $m = |E|$

Example graphs

- Cities connected by direct air flights: node = city, edge = flight
- Social networks: node = person, edge = friend
- An **undirected graph** describes friends on a social network
- When you follow somebody you have an edge from one to another, i.e. a **directed graph**
- Actually, we can view city connectivity through air flights as a directed graph but normally there is a flight back
- Chess games: node = position, edge = legal move

Graph representation: adjacency matrix

- $n = |V|$ and $m = |E|$
- Number each vertex from $1..n$
- Often two representations of each edge
- If there is an edge $i - j$ then one is stored both in $m[i][j]$ and in $m[j][i]$, otherwise a zero
- If n is large it can be a good idea to store only half the matrix
- $\Theta(n^2)$ space
- $\Theta(1)$ time to check if there is an edge $i - j$
- $\Theta(n)$ time to find all neighbors of a node
- $\Theta(n^2)$ time to list all edges

Graph representation: adjacency list

- $n = |V|$ and $m = |E|$
- Every vertex has a list of neighbors
- Every edge $u - v$ is stored in both u and v
- $degree(n)$ is the number of neighbors
- $\Theta(degree(n))$ time to find all neighbors of a node
- $\Theta(m)$ time to list all edges

Optimizations

- Store only half of the adjacency matrix for undirected graphs
- For a very dense graph the matrix is smaller and just as fast
- If you need both quick neighbor check and being able to quickly list all neighbors, then use both!
- Optimizing compilers use both when deciding which variable should be allocated a processor register: the variables are nodes and there is an edge $x - y$ if x and y may be needed at the same time (and therefore cannot use the same register)

Paths and connectivity

- A **path** is a sequence of nodes $p = (v_1, v_2, ..., v_k)$ such that v_i and v_{i+1} are neighbors in an undirected graph, or there is an edge from v_i to v_{i+1} in a directed graph.
- If all nodes in p are distinct then it is a **simple path**.
- An undirected graph is **connected** if there is a path between every pair of nodes.
- A **cycle** is a path which consists of a simple path followed by the first node such as (u, v, w, u).

Trees

- A connected undirected graph is a **tree** if it has no cycle.
- A tree has $n - 1$ edges.
- In a **rooted tree** one node, r is called the root node.

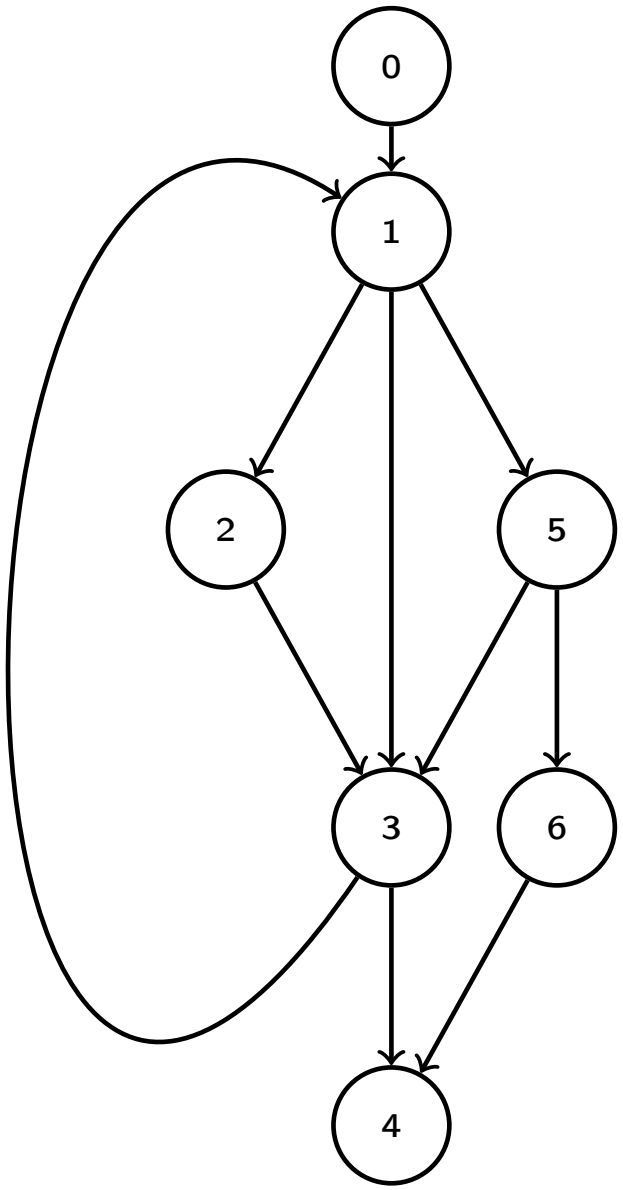
Depth first search: DFS

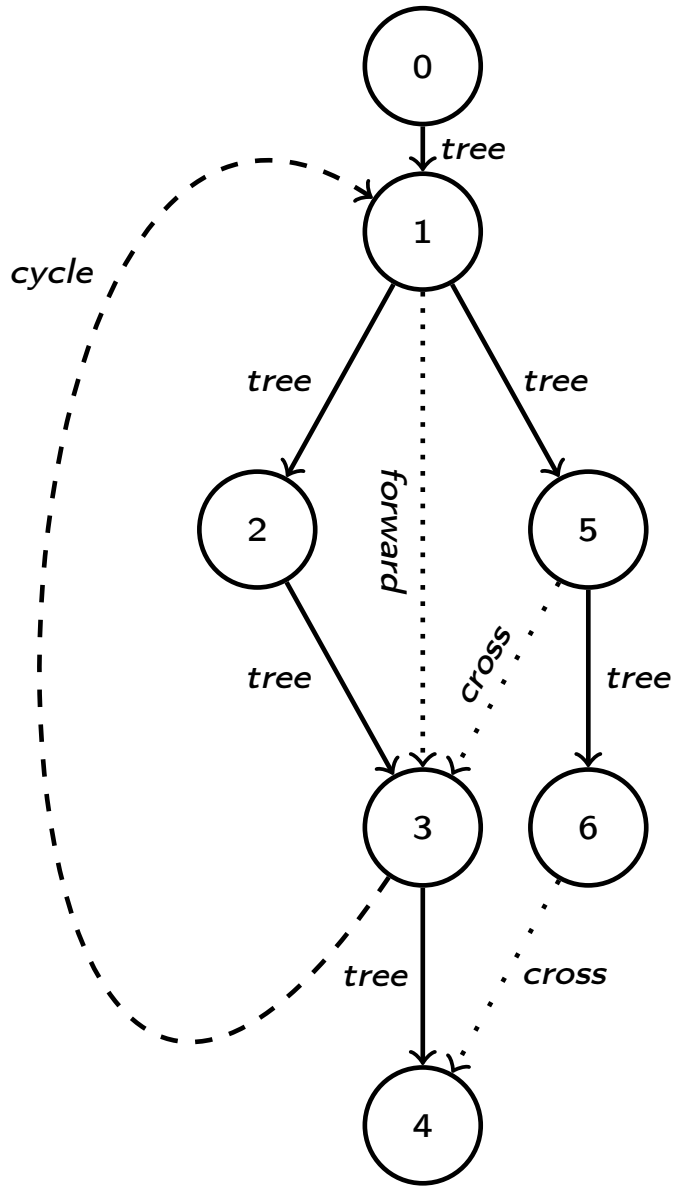
```
int dfnum;                    /* Depth-first search number. */
procedure dfs(v)
begin
    dfn(v) ← dfnum
    visited(v) ← true
    dfnum ← dfnum +1

    for each w ∈ succ(v) do
        if (not visited(w))
            dfs(w)
end
procedure depth_first_search(V)
        dfnum ← 0
        for each v ∈ V do
            visited(v) ← false
        dfs(s)
end
```

DFS

- Properties of depth-first search have been studied extensively by Robert Tarjan
- DFS is used a lot in compilers
- His algorithms tend to be faster than others' and often more beautiful than art

DFS example

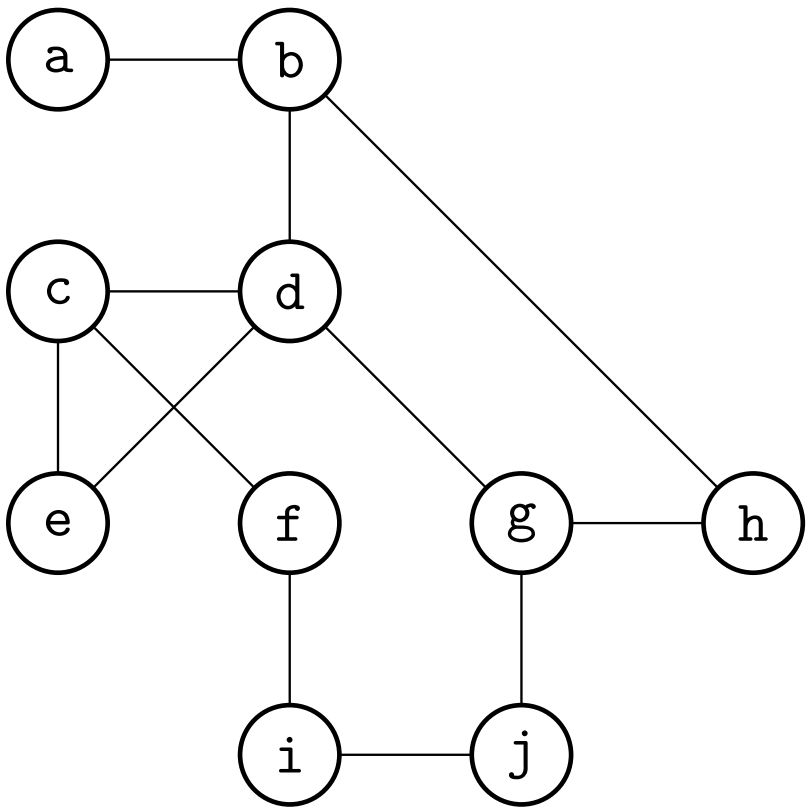

0
1
2
5
3
6
4


0
tree
1
cycle
tree
tree
forward
2
5
tree
cross
tree
3
6
tree
cross
4

The s-t connectivity problem: Lab 1

- The problem is to find a path from s to t.
- Often we want to find the shortest path from s to t.
- The **distance** between two nodes u and v is the number of edges on a shortest path from u to v
- How to solve the connectivity problem?
- Let $k = 1, 2, 3, ..., \infty$
- Check all nodes v at a distance k from s until either $v = t$ or there are no more nodes to check, in which case s and t are not connected.
- This is called **breadth first search**, or simply **BFS**
- Think of an onion. You start in the center and explore one layer at a time outwards.

Breadth first search



- Is there a path from a to j?
- A node v is added to a layer only the first time v is seen
- Check one layer at a time.
- $L_0 = \{a\}$
- $L_1 = \{b\}$
- $L_2 = \{d, h\}$
- $L_3 = \{c, e, g\}$
- $L_4 = \{f, j\}$
- But we don't need the layers.
- A list is sufficient.

BFS implementation to find path $s - t$

```
procedure BFS(G, s, t)
    q ← new list containing s
    for v ∈ V visited(v) ← 0
    visited(s) ← 1
    while q ≠ null
        v ← take out the first element from q
        for w ∈ neighbor(v)
            if not visited(w) then
                visited(w) ← 1
                add w to end of q
                pred(w) ← v
                if w = t then
                    print "found path s − t"
                    return
    print "found no path s − t"
```

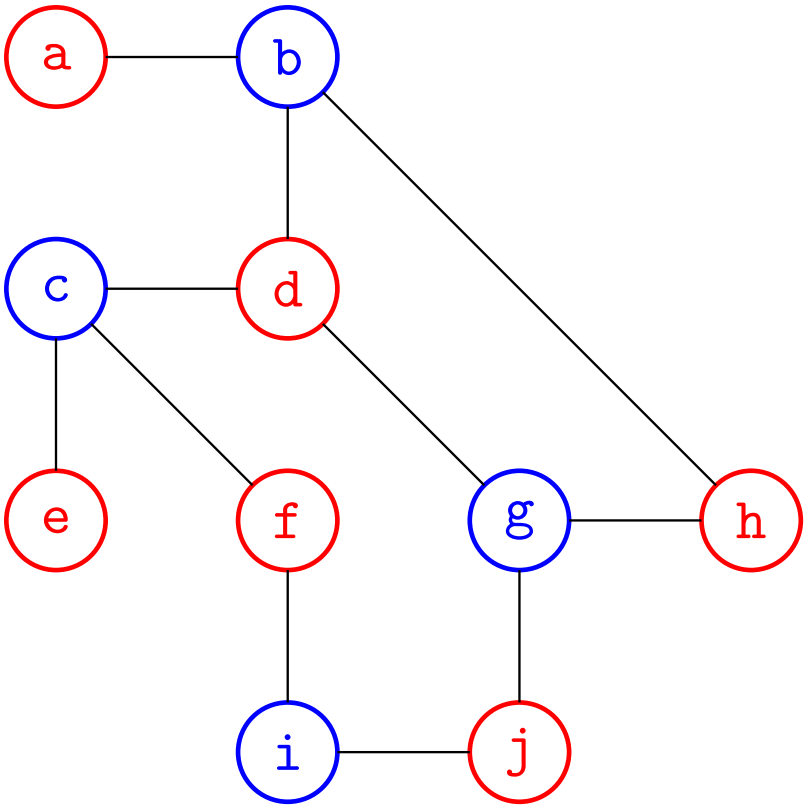
Finding the actual path $s - t$

- We want to find a path $a - j$
- One is $p = (a, b, d, g, j)$
- For each node w except the first, the attribute $pred(w)$ is the previous node in p.
- $pred(j) = g$, $pred(g) = d$, etc
- What is the running time of BFS?
- The while loop has up to n iterations with $|V| = n$
- Each node has at most n neighbors, so $O(n^2)$?
- What do you say?

BFS time complexity

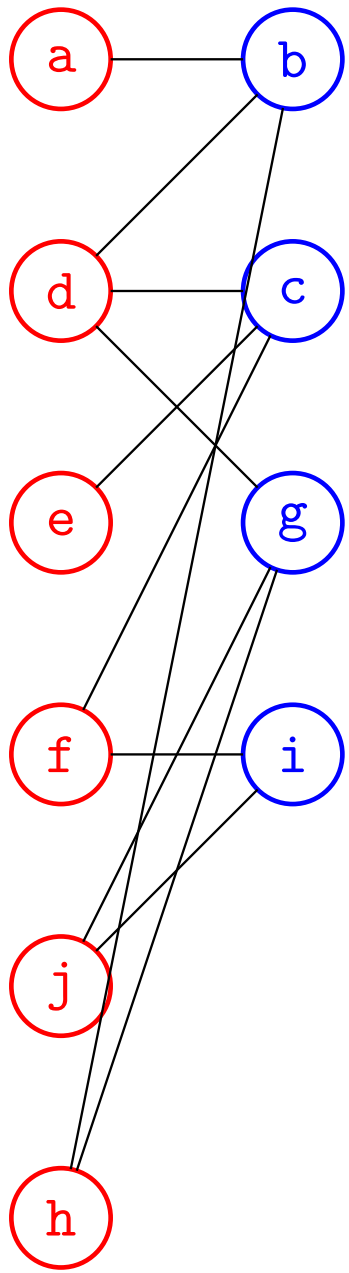
- But in total m edges so $2m = \sum_{v \in V} degree(v)$ edges to process.
- $2m$ since each edge is in two adjacency lists
- Thus BFS can be implemented in $O(n + m)$ with adjacency lists

Bipartite graphs



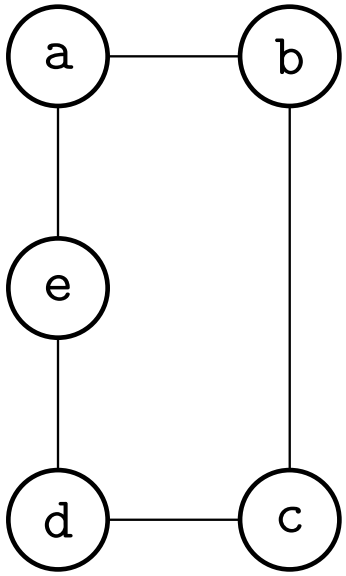
- A graph is **bipartite** if it can be colored with two colors such that every edge connects two nodes with different colors.

Bipartite graphs



- What could prevent a graph from being bipartite?

Bipartite graphs



- A graph with a cycle with an odd length is impossible to color with two colors.

Characterizing bipartite graphs

Lemma

- *Let G be a connected graph and $L_0, L_1, ..., L_k$ the layers of BFS.*
- *Exactly one of the following is true for any G:*
  (1) *There is no edge between nodes in any L_i.*
  (2) *There is an edge between nodes in some L_i.*

  In (1) G is bipartite and in (2) G is not bipartite.

Proof.

- First we show (1):
  - Assume there is no edge $v - w$ such that $\{v, w\} \subseteq L_i$
  - Color each node in an odd layer in one color and in even layers in the other.
  - Thus G is bipartite.

□

Characterizing bipartite graphs

Lemma

- *Let G be a connected graph and $L_0, L_1, ..., L_k$ the layers of BFS.*
- *Exactly one of the following is true for any G:*
  (1) *There is no edge between nodes in any L_i.*
  (2) *There is an edge between nodes in some L_i.*

  In (1) G is bipartite and in (2) G is not bipartite.

Proof.

- Second we show (2):
  - We will show that (2) implies there is an odd length cycle in G.
  - Assume there is an edge $v - w$ such that $\{v, w\} \subseteq L_j$.
  - Let u be their lowest common ancestor and assume $u \in L_i$.
  - There is a cycle $u, ..., v, w, ...u$. The distance between v and w is 1
  - The distance between u and each of v and w is $j - i$
  - The total length of the cycle therefore is $1 + 2(j - i)$ which is odd.
  - Since G contains an odd length cycle, G is not bipartite.

Directed graphs

- Again $G = (V, E)$
- An edge $e = (u, v)$ is from u to v
- Example directed graphs
  - Links from one web page to another
  - Citation from journal article to another
  - Scheduling constraints in projects: some tasks must be performed before others
- BFS: works naturally also for directed graphs

Strong connectivity in directed graphs

- Nodes u and v are **mutually reachable** if there is a path from u to v and a path from v to u
- A directed graph is **strongly connected** if every pair of nodes are mutually reachable

Lemma

Let s be any node in G. G is strongly connected $\iff$ every node is reachable from s and s is reachable from any node.

Proof.

$\Rightarrow$ follows directly from the definition of strongly connected G. $\Leftarrow$ follows by constructing two paths:

- a path from u to v as $p = (u, ..., s, ..., v)$, and
- a path from v to u as $q = (v, ..., s, ..., u)$.

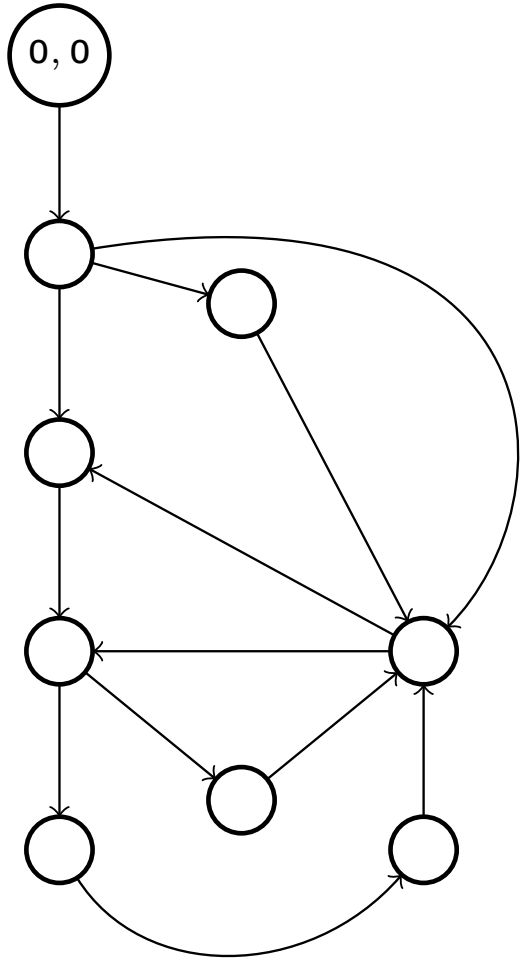
$\square$

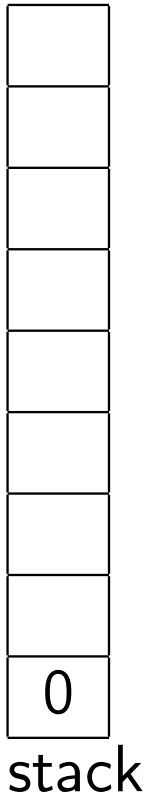
Determining strong connectivity

- Select any node $v \in V$
- Use BFS on G from v and check if all of V is reached
- Construct G^r from G by reversing all edges
- Use BFS on G^r from v and check if all of V is reached
- If all of V is reached in both searches, G is strongly connected
- $O(n + m)$
- We will next see Tarjan's algorithm which instead uses depth first search and also lists all the strongly connected components

Tarjan's Algorithm: Initial Processing of 0

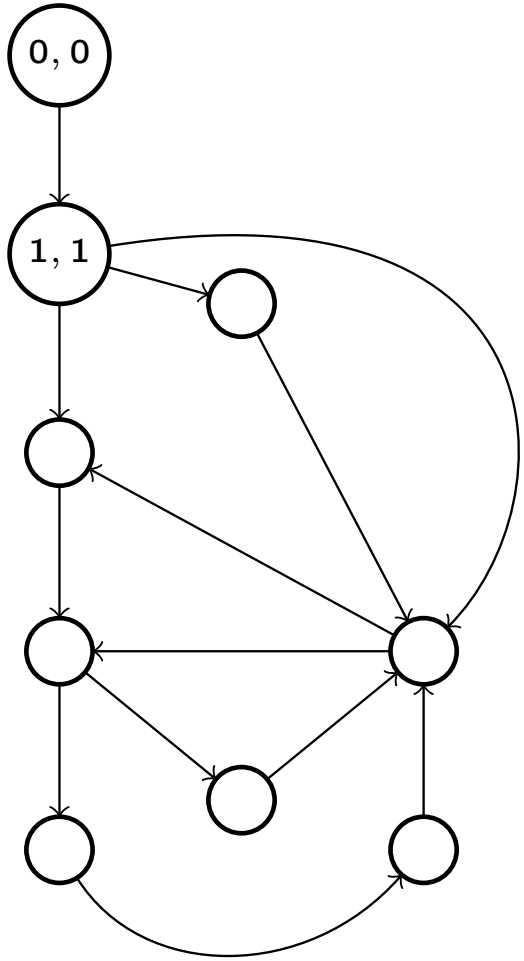
```
int    dfnum

procedure strong_connect(v)
    dfn(v) ← dfnum
    lowlink(v) ← dfnum
    visited(v) ← true
    push(v)
    dfnum ← dfnum +1

    for each w ∈ succ(v) do
        if (not visited(w)) {
            strong_connect(w)
            lowlink(v) ← min(lowlink(v), lowlink(w))
        } else if (dfn(w) < dfn(v)  and w is on stack)
            lowlink(v) ← min(lowlink(v), dfn(w))

    if (lowlink(v) = dfn(v))
        scc ← ∅
        do
            w ← pop()
            add w to scc
        while (w ≠ v)
        process_scc(scc)
end
```

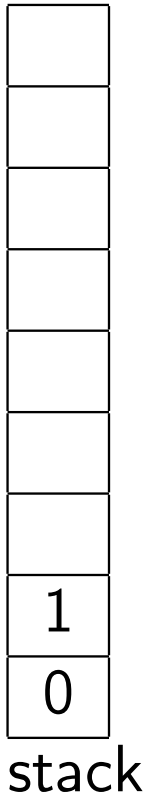




Tarjan's Algorithm: Initial Processing of 1

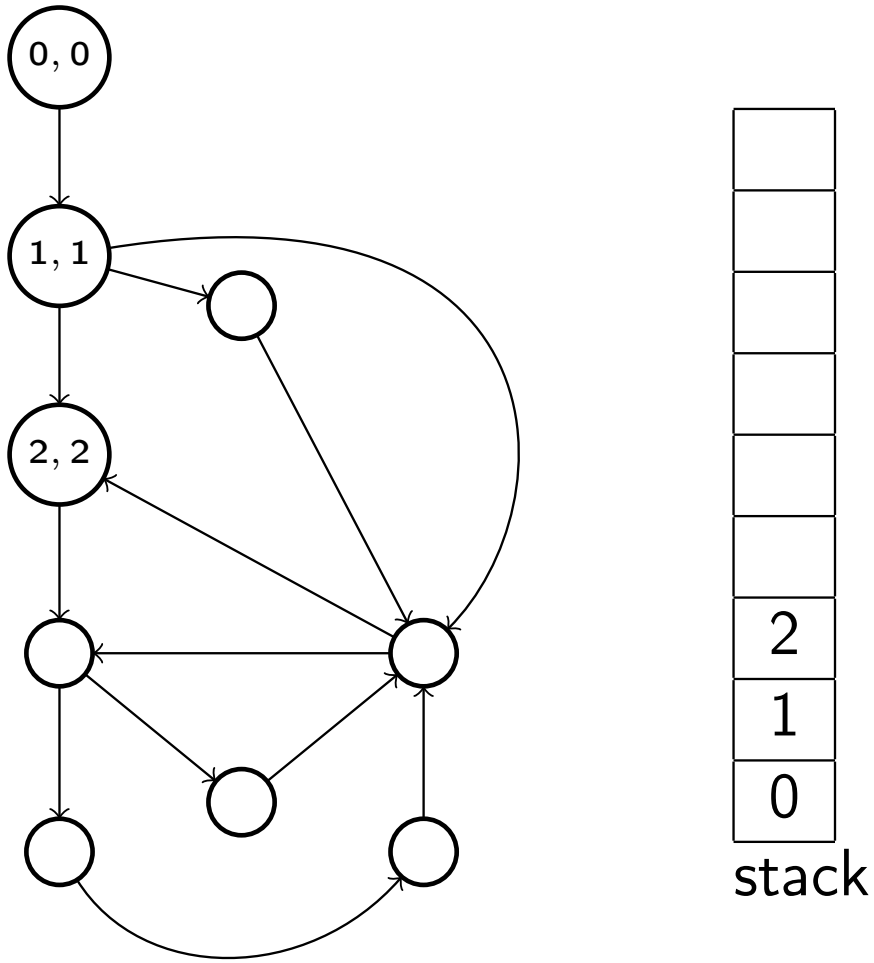
```
int    dfnum

procedure strong_connect(v)
    dfn(v) ← dfnum
    lowlink(v) ← dfnum
    visited(v) ← true
    push(v)
    dfnum ← dfnum +1

    for each w ∈ succ(v) do
        if (not visited(w)) {
            strong_connect(w)
            lowlink(v) ← min(lowlink(v), lowlink(w))
        } else if (dfn(w) < dfn(v)  and w is on stack)
            lowlink(v) ← min(lowlink(v), dfn(w))

    if (lowlink(v) = dfn(v))
        scc ← ∅
        do
            w ← pop()
            add w to scc
        while (w ≠ v)
        process_scc(scc)
end
```





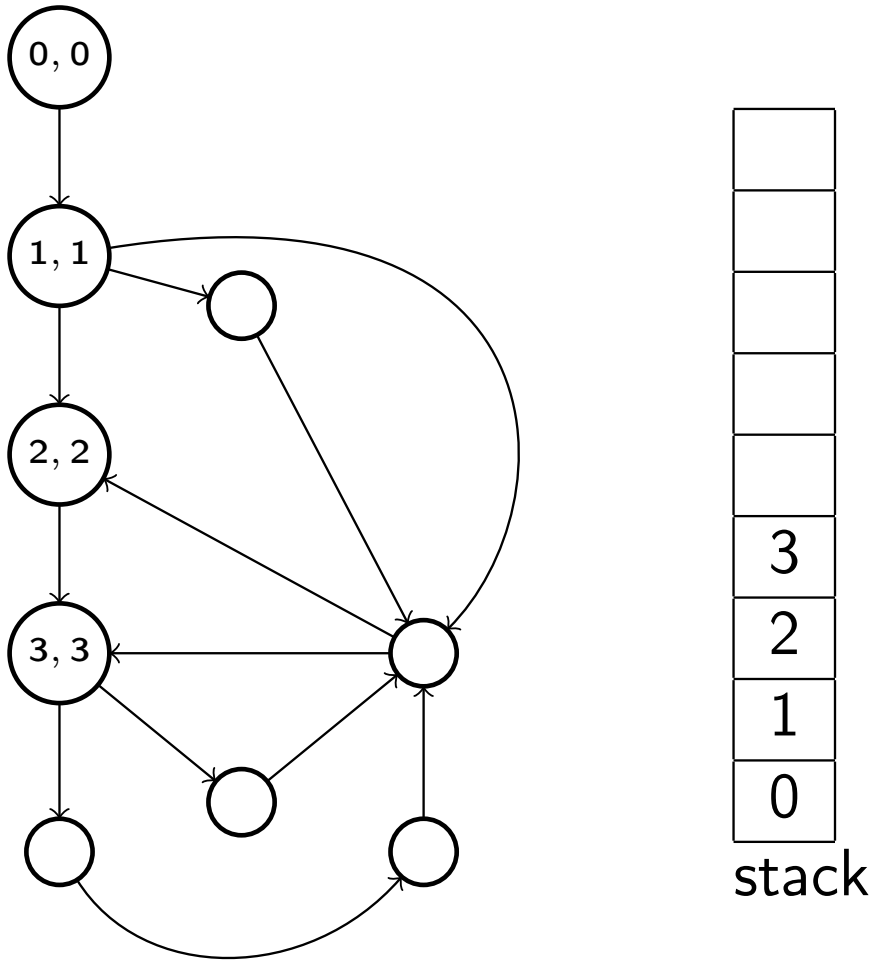
Tarjan's Algorithm: Initial Processing of 2

```
int    dfnum

procedure strong_connect(v)
    dfn(v) ← dfnum
    lowlink(v) ← dfnum
    visited(v) ← true
    push(v)
    dfnum ← dfnum +1

    for each w ∈ succ(v) do
        if (not visited(w)) {
            strong_connect(w)
            lowlink(v) ← min(lowlink(v), lowlink(w))
        } else if (dfn(w) < dfn(v) and w is on stack)
            lowlink(v) ← min(lowlink(v), dfn(w))

    if (lowlink(v) = dfn(v))
        scc ← ∅
        do
            w ← pop()
            add w to scc
        while (w ≠ v)
        process_scc(scc)
end
```



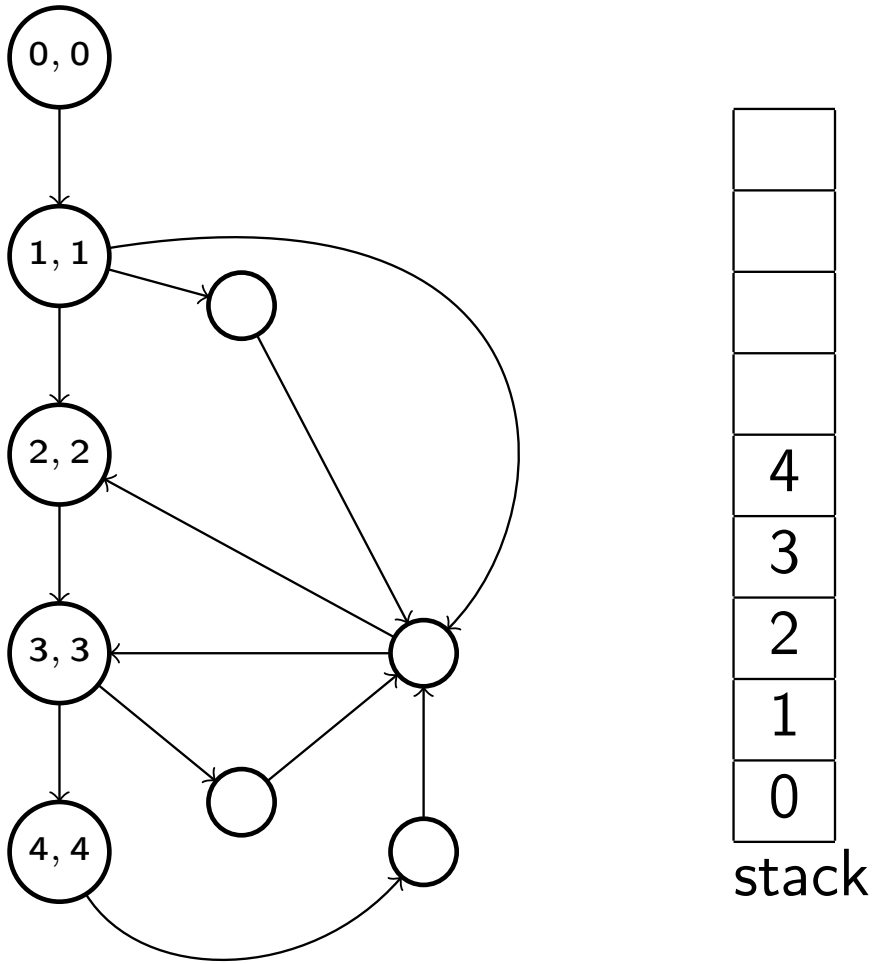
Tarjan's Algorithm: Initial Processing of 3

```
int    dfnum

procedure strong_connect(v)
    dfn(v) ← dfnum
    lowlink(v) ← dfnum
    visited(v) ← true
    push(v)
    dfnum ← dfnum +1

    for each w ∈ succ(v) do
        if (not visited(w)) {
            strong_connect(w)
            lowlink(v) ← min(lowlink(v), lowlink(w))
        } else if (dfn(w) < dfn(v) and w is on stack)
            lowlink(v) ← min(lowlink(v), dfn(w))

    if (lowlink(v) = dfn(v))
        scc ← ∅
        do
            w ← pop()
            add w to scc
        while (w ≠ v)
        process_scc(scc)
end
```



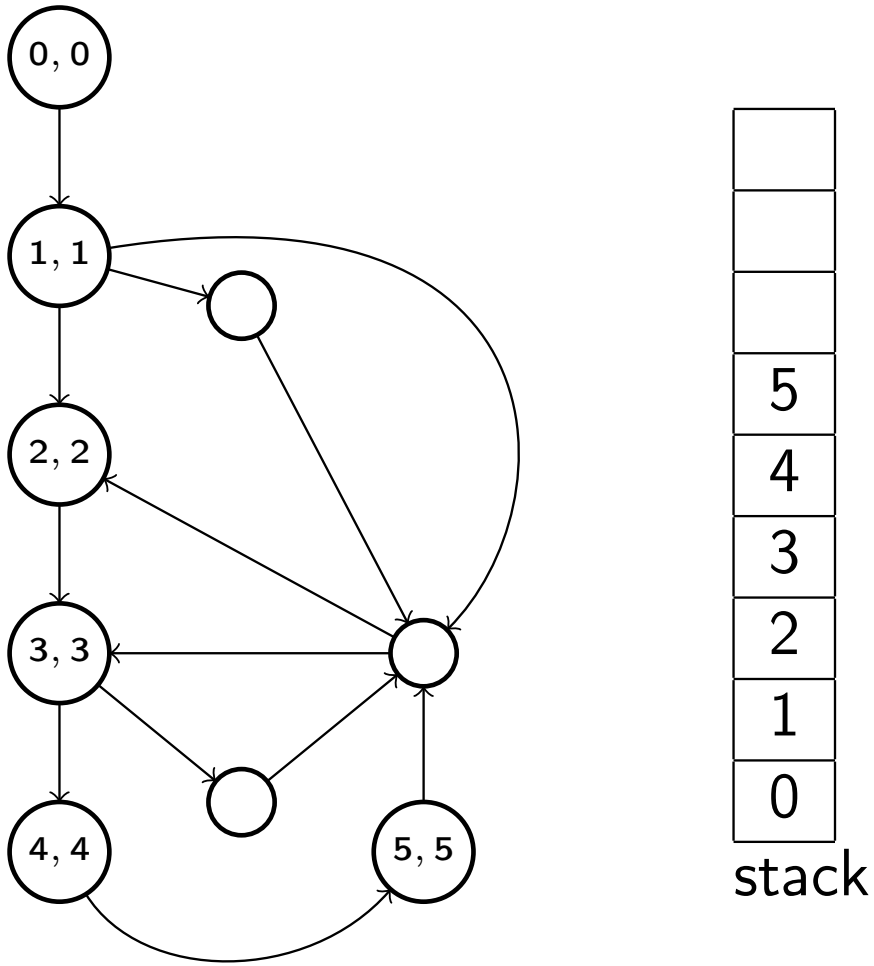
Tarjan's Algorithm: Initial Processing of 4

```
int    dfnum

procedure strong_connect(v)
    dfn(v) ← dfnum
    lowlink(v) ← dfnum
    visited(v) ← true
    push(v)
    dfnum ← dfnum +1

    for each w ∈ succ(v) do
        if (not visited(w)) {
            strong_connect(w)
            lowlink(v) ← min(lowlink(v), lowlink(w))
        } else if (dfn(w) < dfn(v) and w is on stack)
            lowlink(v) ← min(lowlink(v), dfn(w))

    if (lowlink(v) = dfn(v))
        scc ← ∅
        do
            w ← pop()
            add w to scc
        while (w ≠ v)
        process_scc(scc)
end
```



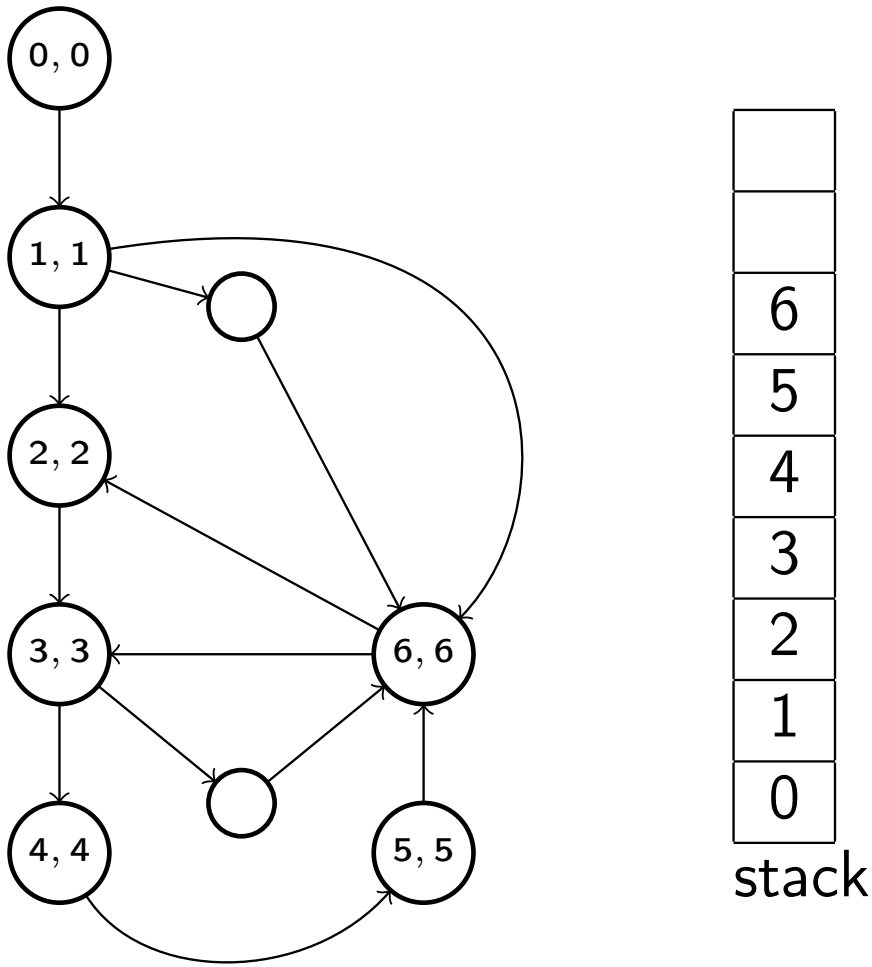
Tarjan's Algorithm: Initial Processing of 5

```
int    dfnum

procedure strong_connect(v)
    dfn(v) ← dfnum
    lowlink(v) ← dfnum
    visited(v) ← true
    push(v)
    dfnum ← dfnum +1

    for each w ∈ succ(v) do
        if (not visited(w)) {
            strong_connect(w)
            lowlink(v) ← min(lowlink(v), lowlink(w))
        } else if (dfn(w) < dfn(v) and w is on stack)
            lowlink(v) ← min(lowlink(v), dfn(w))

    if (lowlink(v) = dfn(v))
        scc ← ∅
        do
            w ← pop()
            add w to scc
        while (w ≠ v)
        process_scc(scc)
end
```



Tarjan's Algorithm: Initial Processing of 6

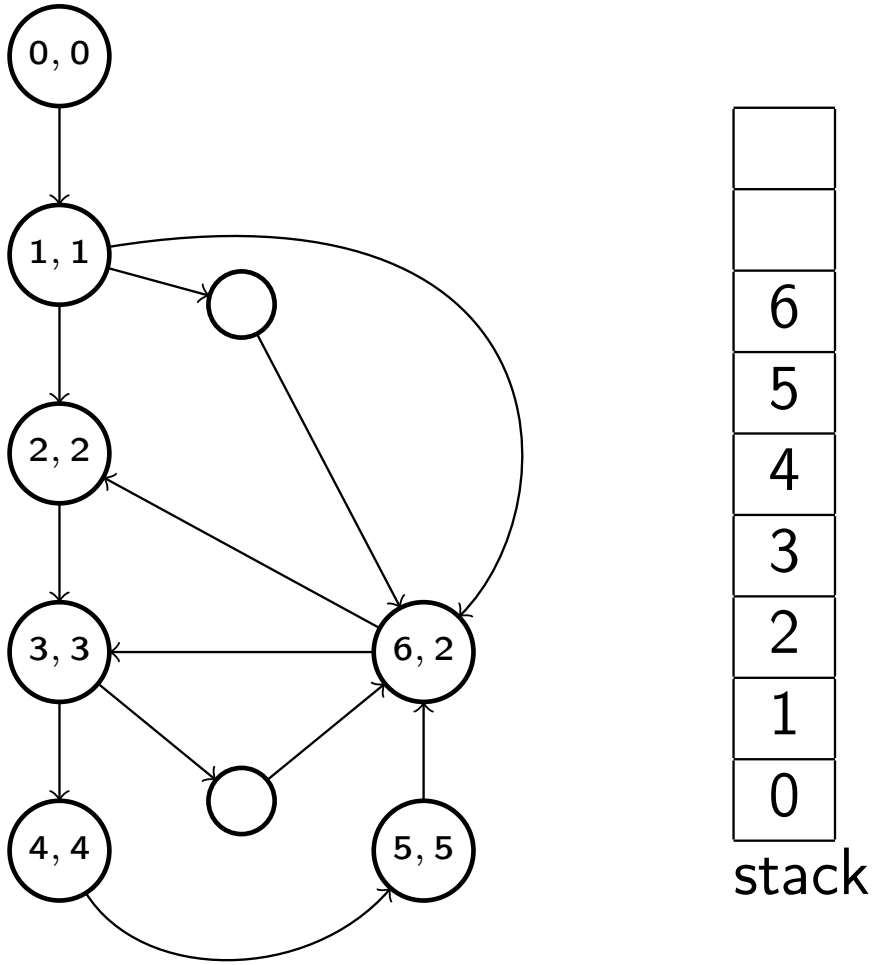
```
int    dfnum

procedure strong_connect(v)
    dfn(v) ← dfnum
    lowlink(v) ← dfnum
    visited(v) ← true
    push(v)
    dfnum ← dfnum +1

    for each w ∈ succ(v) do
        if (not visited(w)) {
            strong_connect(w)
            lowlink(v) ← min(lowlink(v), lowlink(w))
        } else if (dfn(w) < dfn(v)  and w is on stack)
            lowlink(v) ← min(lowlink(v), dfn(w))

    if (lowlink(v) = dfn(v))
        scc ← ∅
        do
            w ← pop()
            add w to scc
        while (w ≠ v)
        process_scc(scc)
end
```



Tarjan's Algorithm: More Processing of 6

- $(6, 2) \Rightarrow 6$ in same scc as 2.

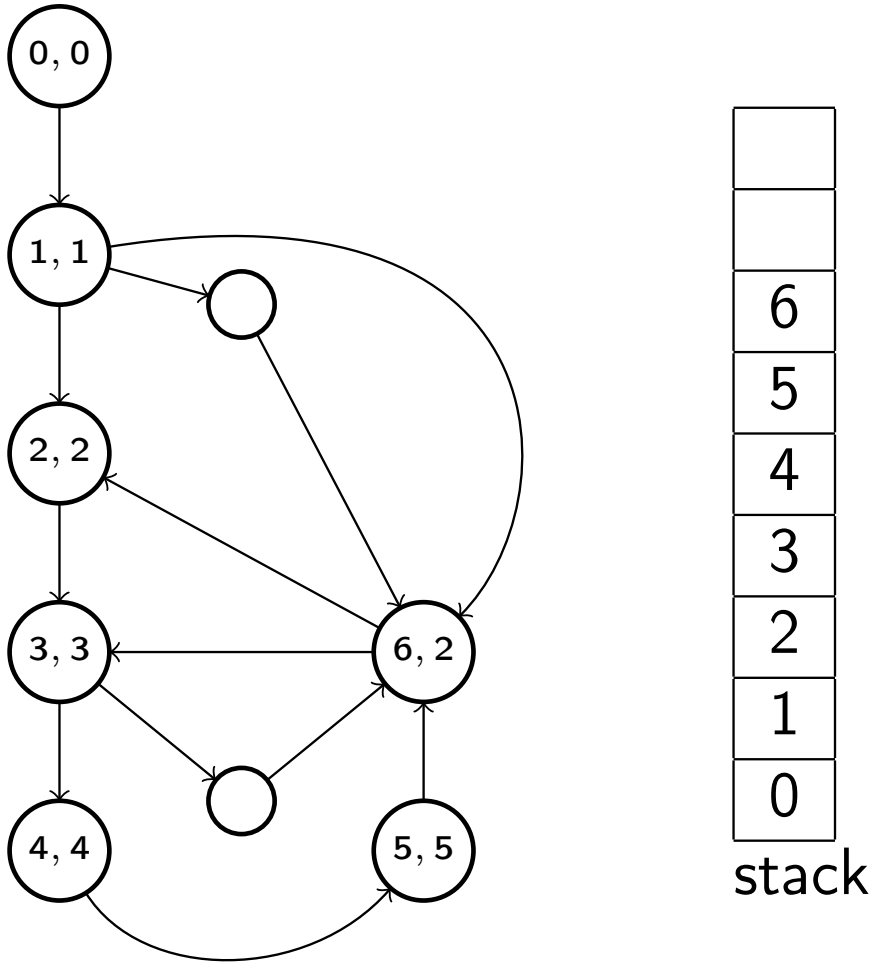
```
int    dfnum

procedure strong_connect(v)
    dfn(v) ← dfnum
    lowlink(v) ← dfnum
    visited(v) ← true
    push(v)
    dfnum ← dfnum +1

    for each w ∈ succ(v) do
        if (not visited(w)) {
            strong_connect(w)
            lowlink(v) ← min(lowlink(v), lowlink(w))
        } else if (dfn(w) < dfn(v)  and w is on stack)
            lowlink(v) ← min(lowlink(v), dfn(w))

    if (lowlink(v) = dfn(v))
        scc ← ∅
        do
            w ← pop()
            add w to scc
        while (w ≠ v)
        process_scc(scc)
end
```



Tarjan's Algorithm: More Processing of 6

- (6, 3). no action.

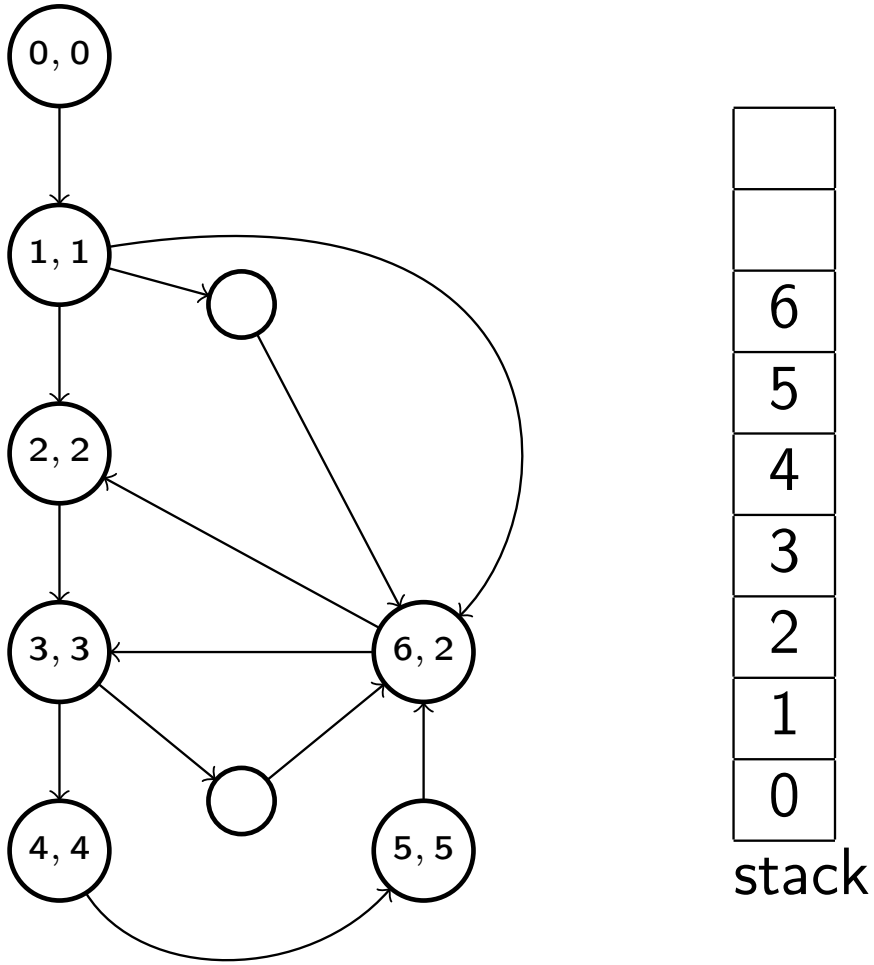
```
int    dfnum

procedure strong_connect(v)
    dfn(v) ← dfnum
    lowlink(v) ← dfnum
    visited(v) ← true
    push(v)
    dfnum ← dfnum +1

    for each w ∈ succ(v) do
        if (not visited(w)) {
            strong_connect(w)
            lowlink(v) ← min(lowlink(v), lowlink(w))
        } else if (dfn(w) < dfn(v) and w is on stack)
            lowlink(v) ← min(lowlink(v), dfn(w))

    if (lowlink(v) = dfn(v))
        scc ← ∅
        do
            w ← pop()
            add w to scc
        while (w ≠ v)
        process_scc(scc)
end
```



Tarjan's Algorithm: More Processing of 6

- 6 remains on the stack.

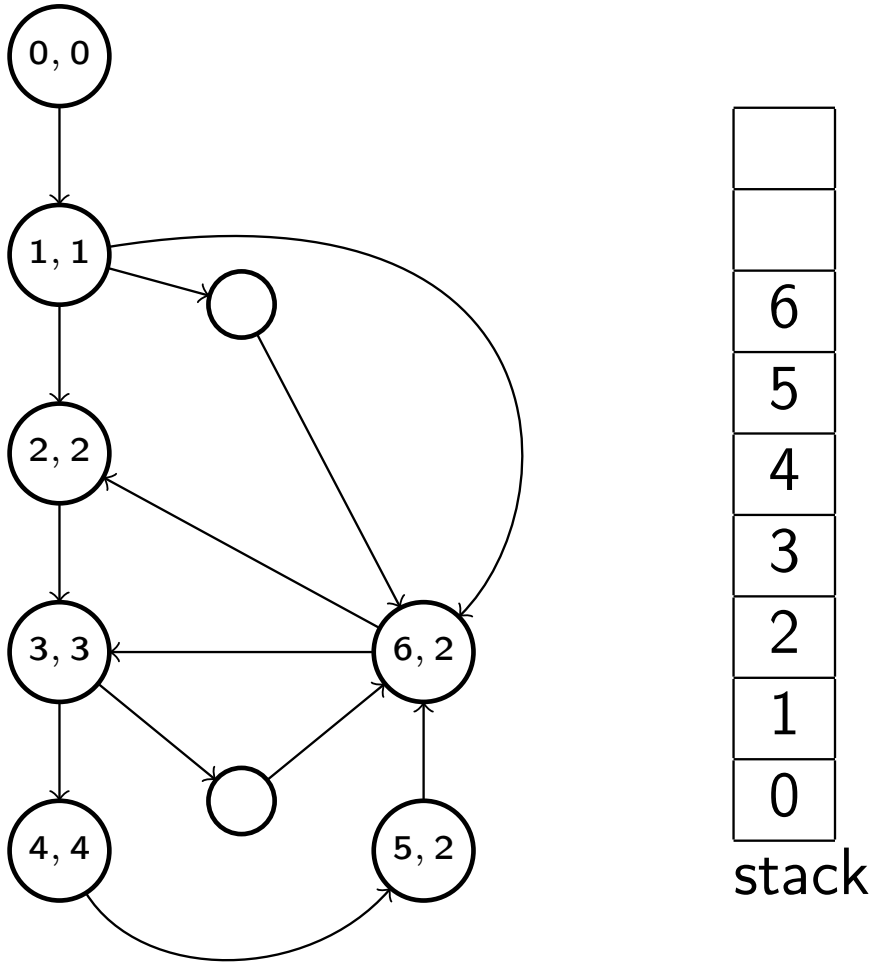
```
int    dfnum

procedure strong_connect(v)
    dfn(v) ← dfnum
    lowlink(v) ← dfnum
    visited(v) ← true
    push(v)
    dfnum ← dfnum +1

    for each w ∈ succ(v) do
        if (not visited(w)) {
            strong_connect(w)
            lowlink(v) ← min(lowlink(v), lowlink(w))
        } else if (dfn(w) < dfn(v)  and w is on stack)
            lowlink(v) ← min(lowlink(v), dfn(w))

    if (lowlink(v) = dfn(v))
        scc ← ∅
        do
            w ← pop()
            add w to scc
        while (w ≠ v)
        process_scc(scc)
end
```



Tarjan's Algorithm: More Processing of 5

- New lowlink and remains.

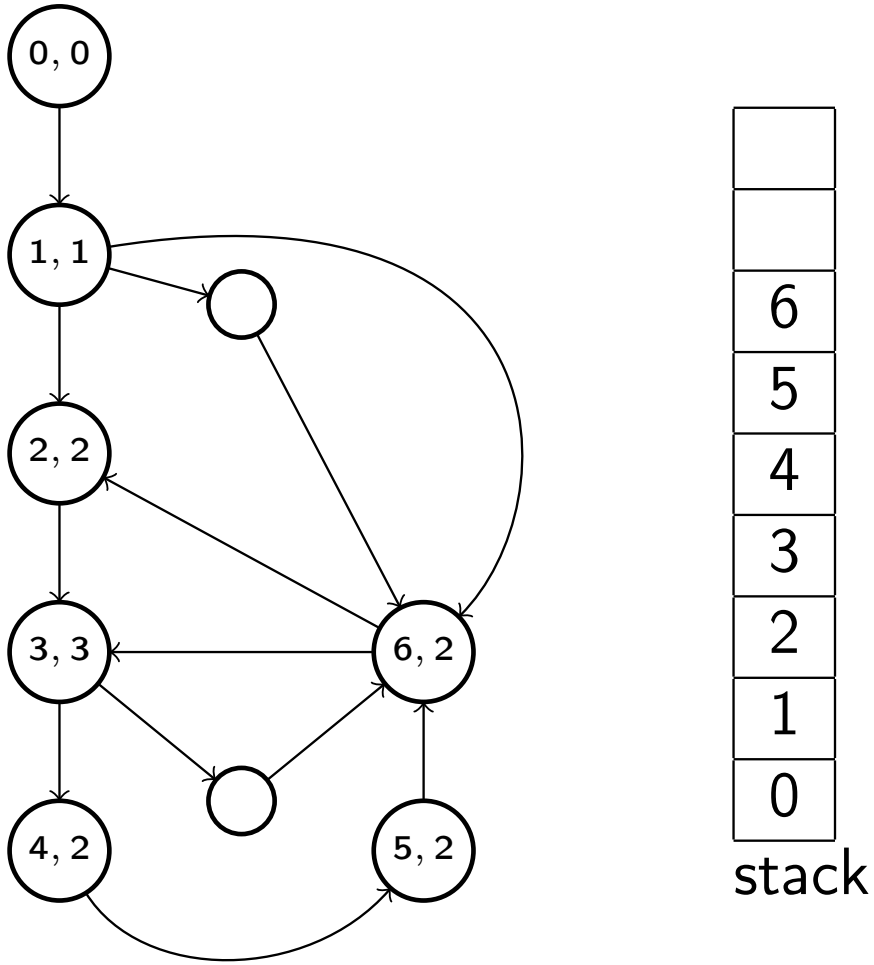
```
int    dfnum

procedure strong_connect(v)
    dfn(v) ← dfnum
    lowlink(v) ← dfnum
    visited(v) ← true
    push(v)
    dfnum ← dfnum +1

    for each w ∈ succ(v) do
        if (not visited(w)) {
            strong_connect(w)
            lowlink(v) ← min(lowlink(v), lowlink(w))
        } else if (dfn(w) < dfn(v)  and w is on stack)
            lowlink(v) ← min(lowlink(v), dfn(w))

    if (lowlink(v) = dfn(v))
        scc ← ∅
        do
            w ← pop()
            add w to scc
        while (w ≠ v)
        process_scc(scc)
end
```



Tarjan's Algorithm: More Processing of 4

- New lowlink and remains.

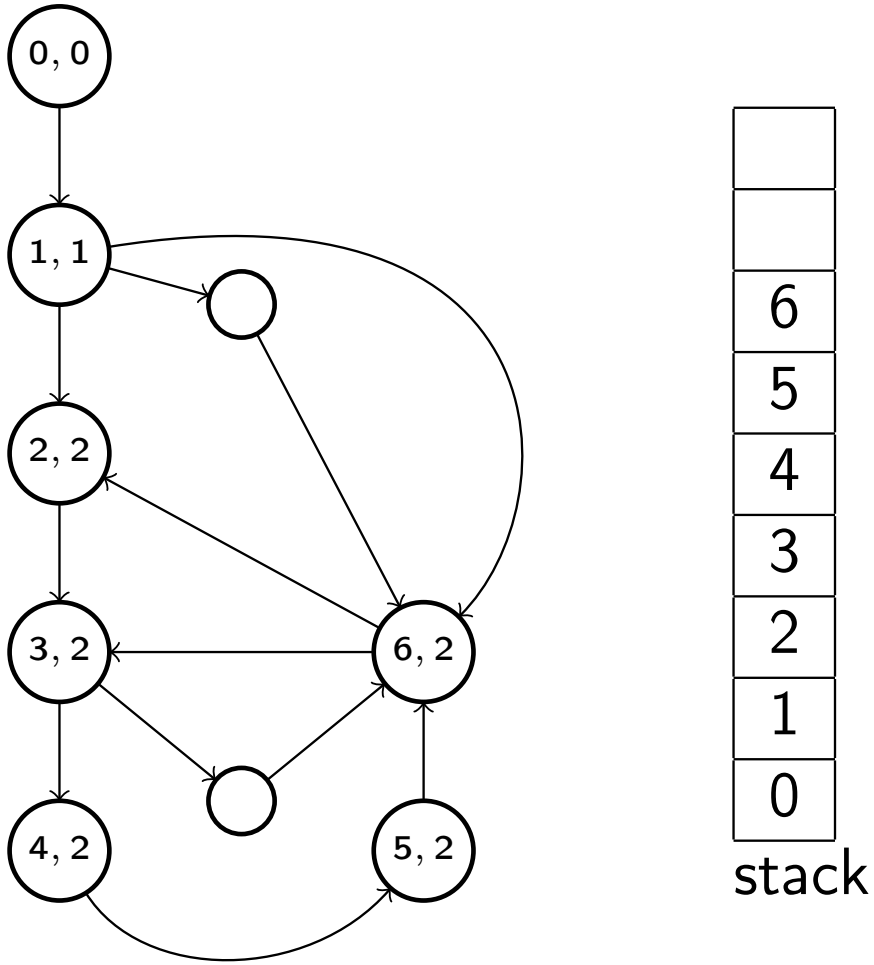
```
int    dfnum

procedure strong_connect(v)
    dfn(v) ← dfnum
    lowlink(v) ← dfnum
    visited(v) ← true
    push(v)
    dfnum ← dfnum +1

    for each w ∈ succ(v) do
        if (not visited(w)) {
            strong_connect(w)
            lowlink(v) ← min(lowlink(v), lowlink(w))
        } else if (dfn(w) < dfn(v)  and w is on stack)
            lowlink(v) ← min(lowlink(v), dfn(w))

    if (lowlink(v) = dfn(v))
        scc ← ∅
        do
            w ← pop()
            add w to scc
        while (w ≠ v)
        process_scc(scc)
end
```



Tarjan's Algorithm: More Processing of 3

- New lowlink. Next 7.

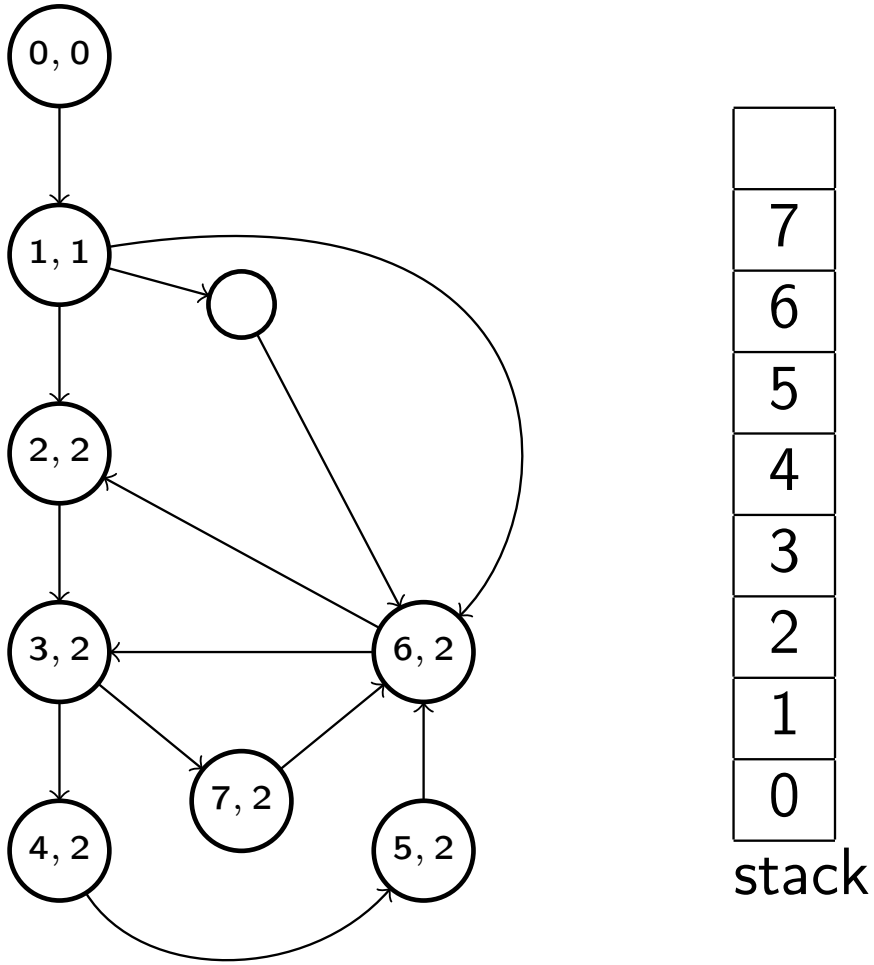
```
int    dfnum

procedure strong_connect(v)
    dfn(v) ← dfnum
    lowlink(v) ← dfnum
    visited(v) ← true
    push(v)
    dfnum ← dfnum +1

    for each w ∈ succ(v) do
        if (not visited(w)) {
            strong_connect(w)
            lowlink(v) ← min(lowlink(v), lowlink(w))
        } else if (dfn(w) < dfn(v)  and w is on stack)
            lowlink(v) ← min(lowlink(v), dfn(w))

    if (lowlink(v) = dfn(v))
        scc ← ∅
        do
            w ← pop()
            add w to scc
        while (w ≠ v)
        process_scc(scc)
end
```



Tarjan's Algorithm: Processing of 7

- Lowlink is set.

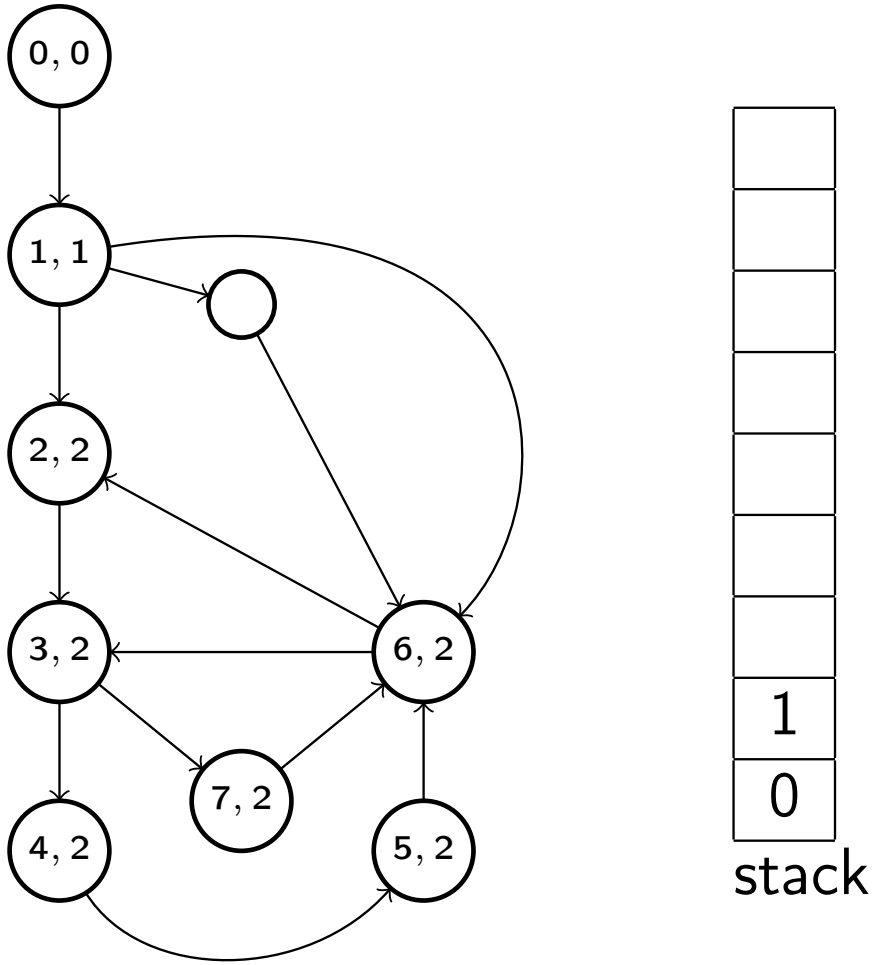
```
int    dfnum

procedure strong_connect(v)
    dfn(v) ← dfnum
    lowlink(v) ← dfnum
    visited(v) ← true
    push(v)
    dfnum ← dfnum +1

    for each w ∈ succ(v) do
        if (not visited(w)) {
            strong_connect(w)
            lowlink(v) ← min(lowlink(v), lowlink(w))
        } else if (dfn(w) < dfn(v)  and w is on stack)
            lowlink(v) ← min(lowlink(v), dfn(w))

    if (lowlink(v) = dfn(v))
        scc ← ∅
        do
            w ← pop()
            add w to scc
        while (w ≠ v)
        process_scc(scc)
end
```



Tarjan's Algorithm: More Processing of 2

- Remove SCC from stack

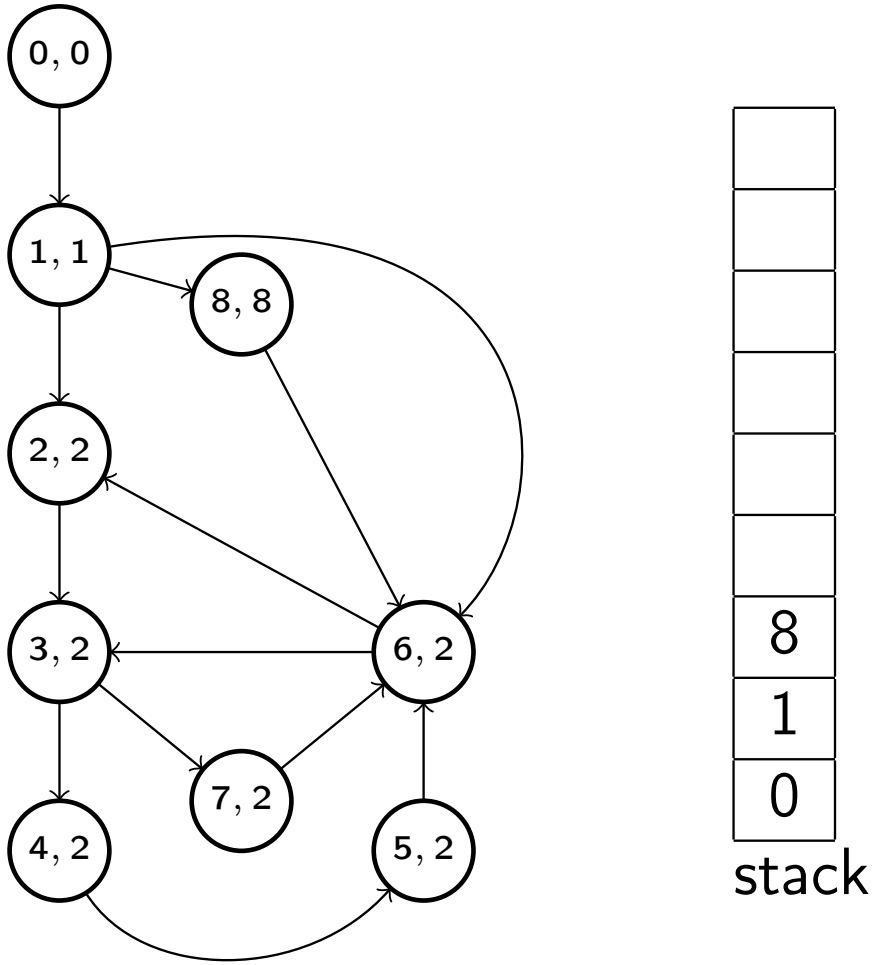
```
int    dfnum

procedure strong_connect(v)
    dfn(v) ← dfnum
    lowlink(v) ← dfnum
    visited(v) ← true
    push(v)
    dfnum ← dfnum +1

    for each w ∈ succ(v) do
        if (not visited(w)) {
            strong_connect(w)
            lowlink(v) ← min(lowlink(v), lowlink(w))
        } else if (dfn(w) < dfn(v)  and w is on stack)
            lowlink(v) ← min(lowlink(v), dfn(w))

    if (lowlink(v) = dfn(v))
        scc ← ∅
        do
            w ← pop()
            add w to scc
        while (w ≠ v)
        process_scc(scc)
end
```



Tarjan's Algorithm: Initial Processing of 8

- No path from 2 to 8.

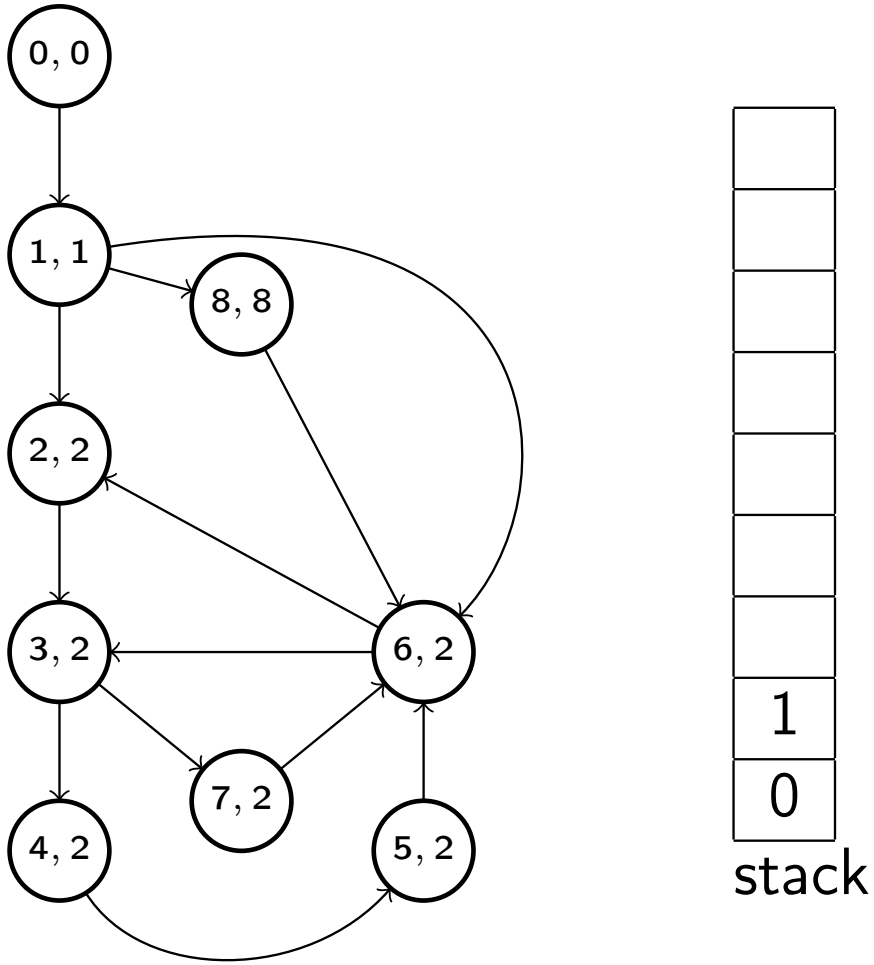
```
int    dfnum

procedure strong_connect(v)
    dfn(v) ← dfnum
    lowlink(v) ← dfnum
    visited(v) ← true
    push(v)
    dfnum ← dfnum +1

    for each w ∈ succ(v) do
        if (not visited(w)) {
            strong_connect(w)
            lowlink(v) ← min(lowlink(v), lowlink(w))
        } else if (dfn(w) < dfn(v)  and w is on stack)
            lowlink(v) ← min(lowlink(v), dfn(w))

    if (lowlink(v) = dfn(v))
        scc ← ∅
        do
            w ← pop()
            add w to scc
        while (w ≠ v)
        process_scc(scc)
end
```



Tarjan's Algorithm: More Processing of 8

- 8 is its own SCC.

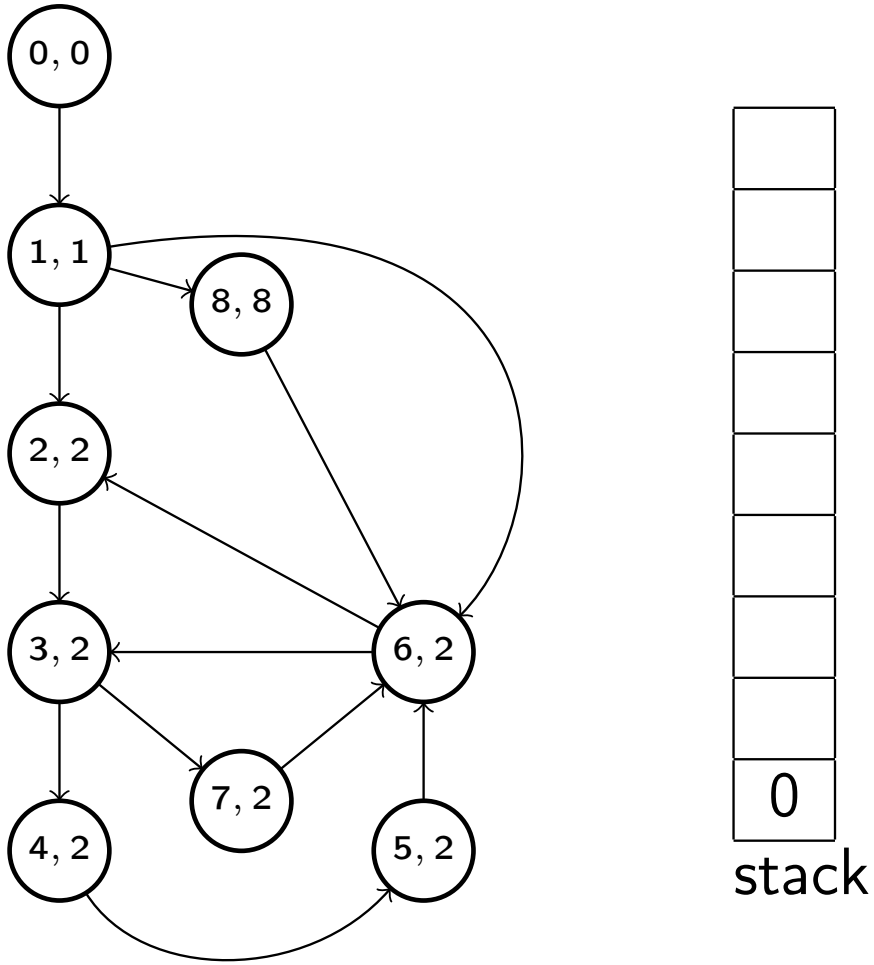
```
int    dfnum

procedure strong_connect(v)
    dfn(v) ← dfnum
    lowlink(v) ← dfnum
    visited(v) ← true
    push(v)
    dfnum ← dfnum +1

    for each w ∈ succ(v) do
        if (not visited(w)) {
            strong_connect(w)
            lowlink(v) ← min(lowlink(v), lowlink(w))
        } else if (dfn(w) < dfn(v)  and w is on stack)
            lowlink(v) ← min(lowlink(v), dfn(w))

    if (lowlink(v) = dfn(v))
        scc ← ∅
        do
            w ← pop()
            add w to scc
        while (w ≠ v)
        process_scc(scc)
end
```



Tarjan's Algorithm: More Processing of 1

- 1 is its own SCC.

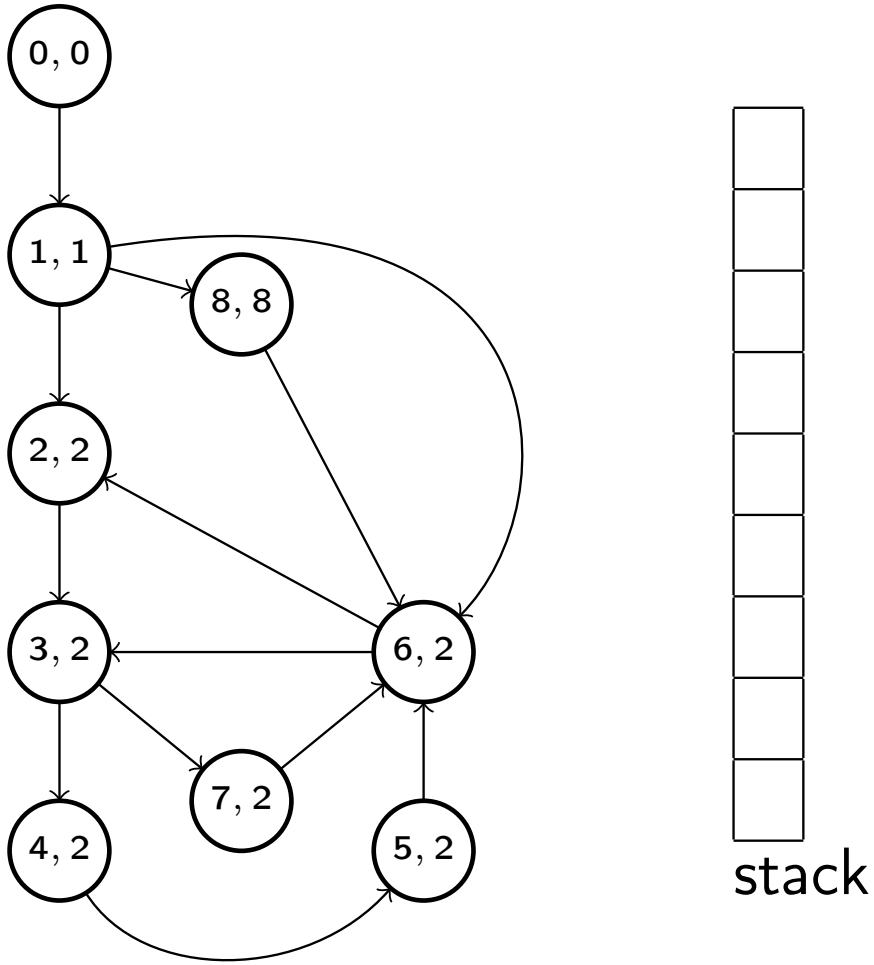
```
int    dfnum

procedure strong_connect(v)
    dfn(v) ← dfnum
    lowlink(v) ← dfnum
    visited(v) ← true
    push(v)
    dfnum ← dfnum +1

    for each w ∈ succ(v) do
        if (not visited(w)) {
            strong_connect(w)
            lowlink(v) ← min(lowlink(v), lowlink(w))
        } else if (dfn(w) < dfn(v)  and w is on stack)
            lowlink(v) ← min(lowlink(v), dfn(w))

    if (lowlink(v) = dfn(v))
        scc ← ∅
        do
            w ← pop()
            add w to scc
        while (w ≠ v)
        process_scc(scc)
end
```



Tarjan's Algorithm: More Processing of 0

- 0 is its own SCC.

```
int    dfnum

procedure strong_connect(v)
    dfn(v) ← dfnum
    lowlink(v) ← dfnum
    visited(v) ← true
    push(v)
    dfnum ← dfnum +1

    for each w ∈ succ(v) do
        if (not visited(w)) {
            strong_connect(w)
            lowlink(v) ← min(lowlink(v), lowlink(w))
        } else if (dfn(w) < dfn(v)  and w is on stack)
            lowlink(v) ← min(lowlink(v), dfn(w))

    if (lowlink(v) = dfn(v))
        scc ← ∅
        do
            w ← pop()
            add w to scc
        while (w ≠ v)
        process_scc(scc)
end
```



Tarjan's Algorithm: Remarks

- Consider the edge (v, w).
- When w is not yet visited we must visit it by calling $strong_connect(w)$.
- If w has been visited, we have two main cases:
  1. w is not on the stack, because it has already found its SCC.
  2. w is on the stack, because it's waiting for being popped.
     - If $dfn(w) < dfn(v)$ then v must set its lowlink so it does not think it is its own SCC.
     - If $dfn(w) \geq dfn(v)$ then no more information for v is available. There is another path from v to w due to which they will belong to the same SCC.

Lab 1: Breadth-first search

- Lab 1 is called Word Ladders
- Two input files are to be read by the program
- The first file is a list of five character words
- The second file contains two words per line
- You should create a directed graph from the first file
- Each word is a node, and there is an edge from v to w if each of the last four characters of v appear in w.
- All four last characters in v must appear with repetition in w
- T<u>HERE</u> should have an edge to ETHER but not to RETCH (since one E is missing).
- BFS should be implemented in $O(n + m)$ but constructing the graph is allowed to be slower